PROJECT REPORT :

Encrypted File Transfer& Secure File Storage

1. Introduction

Encrypted File Transfer and Secure File Storage is a cybersecurity project designed to protect files during transfer and storage. It uses encryption to keep file contents private and HMAC to verify file integrity.

The project is developed using Python. It demonstrates how files can be divided into smaller chunks, encrypted, transferred to a server, and decrypted after successful integrity verification.

2. Objectives

The main objectives of this project are:

- To implement secure file transfer using AES-256 encryption.
- To divide large files into smaller chunks.
- To verify file integrity using HMAC.
- To store encrypted files on the server.
- To understand Man-in-the-Middle (MITM) attacks.
- To understand secure key management.

3. Tools and Technologies

The following tools and technologies are used:

- **Python:** Main programming language.
- **AES-256-GCM:** Used to encrypt and decrypt file data and provide authenticated encryption.
- **HMAC-SHA256:** Used to verify the integrity and authenticity of transferred data.
- **Flask:** Used to create the local server.
- **Requests:** Used for communication between the client and server.

- **Google Colab:** Used as the development and testing environment.

4. Working Procedure

The project works through the following steps:

Step 1: Key Generation

A 256-bit AES encryption key and a separate HMAC key are generated. These keys are used for encryption, decryption, and integrity verification.

Step 2: File Chunking

The selected file is divided into smaller chunks of 1 MB each. This makes it easier to handle large files.

Step 3: File Encryption

Each chunk is encrypted using AES-256-GCM before being transferred to the server. A unique nonce is generated for each chunk.

Step 4: Secure File Transfer

The encrypted chunks are sent from the client to a local Flask server using HTTP requests.

Step 5: Secure Storage

The server stores the encrypted chunks and their related information in a storage directory.

Step 6: Integrity Verification

HMAC-SHA256 is used to verify that the encrypted data has not been modified. The HMAC is checked before decryption.

Step 7: File Decryption

After successful integrity verification, the encrypted chunks are decrypted and combined to recreate the original file.

5. Implementation

The project is implemented in Python using the following main components:

- AES-GCM encryption and decryption.
- File chunking and reassembly.

- HMAC generation and verification.
- Flask-based local server.
- Client-side file upload and download.
- Automated integrity testing.

The program also compares the original file with the downloaded file to check whether the data remains unchanged.

6. Testing and Results

The project includes the following test cases:

Test Case	Purpose
Normal Response	Checks normal file processing.
File Upload	Checks whether encrypted chunks are sent to the server.
File Download	Checks whether the file can be downloaded and decrypted.
Integrity Verification	Checks whether the HMAC verification works.
Tampering Test	Checks whether modified encrypted data is rejected.

Expected Result: The original and downloaded files should match when the transfer and verification are successful. Modified data should fail the integrity verification.

Note: Insert your actual Google Colab output or screenshots here after running the tests.

7. Threat Model

Man-in-the-Middle (MITM) Attack

A Man-in-the-Middle attack occurs when an attacker secretly intercepts or modifies communication between two parties.

AES encryption helps protect file confidentiality, while HMAC helps detect unauthorized changes to transferred data.

However, the current demonstration uses local HTTP communication. It does not implement TLS/HTTPS, so it should not be considered protected against all network-based MITM attacks.

Key Management

Key management means securely generating, storing, and protecting encryption keys.

In this project, AES and HMAC keys are generated in memory. If the runtime session is reset, these keys are lost. A real-world system would require secure key storage, access control, and a safe key backup or recovery process.

8. Limitations

- The project is an educational prototype.
- It runs on a local server rather than a public network.
- HTTPS/TLS is not implemented in the Colab demonstration.
- User authentication and access control are not implemented.
- Keys are stored in memory and are not persistent.
- It is not intended for production use.

9. Conclusion

This project demonstrates the basic concepts of secure file transfer and encrypted storage using Python. AES-256-GCM is used to encrypt file chunks, while HMAC-SHA256 verifies data integrity. File chunking makes it possible to process large files in smaller parts.

The project also provides an understanding of Man-in-the-Middle attacks and the importance of secure key management. Overall, it provides practical knowledge of encryption, data integrity, and basic cybersecurity concepts.

.....

.....